Sales Analytics Pipeline

Phase 1 — Architecture Design

Bronze → Silver → Gold → Export Architecture on Snowflake, aligned to the provided Snowflake Data Architecture Model

Prepared by: Jerry Wyatt

Date: August 21, 2026

Revision: v4 — revises Sections 4 and 8 to describe the Bronze ingestion and orchestration approach actually built for Phase 4 (real Postgres/Close CRM data), and updates Section 10 to close out the outcome-code, role/status-field, and Custom Payment Plan open items now confirmed against real data; see those sections for details

Status: Draft — pending SME review and sign-off before Phase 2 / Phase 3

1. Purpose & Scope

This document proposes the end-to-end architecture for the Sales Analytics Pipeline: how raw CRM activity data lands in Snowflake, how it is repaired and standardized, and how it is modeled to power the four required KPI reports (Inbound Setter, Outbound Setter, Closer, and Objections Faced).

Per the project requirements, this is a Phase 1 proposal covering three areas:

- How JSON is ingested from S3 into Bronze tables.
- How nested structures are flattened, JSON is repaired, and MD5 hashes are calculated for the Silver layer.
- How Dimensions & Facts (Gold views) are designed to serve the four reporting models.

Constraints: no Spark/PySpark, Databricks, or dbt — the pipeline is built natively in Snowflake. This architecture must be verified with the SME before Phase 2 (data modeling / ERD) and Phase 3 (development & deployment) proceed.

This version reconciles the proposed design against the Snowflake Data Architecture Model already provided (table/view names and exact column structures for Bronze, Silver, Gold, and Export). Per SME guidance, that model is a guide rather than a fixed spec — as the pipeline evolves, this design extends it where it makes sense (see the leads_raw decision in Section 4.1) and calls out remaining open items in Section 10.

2. Source Systems Recap

Source tables are extracted daily from the PostgreSQL source (schema: raw), refreshed by 07:30 AM EST, and delivered as nested JSON with single-quoted inner strings:

Source Table	Contents	Key Challenge
leads_raw	Lead info, contacts, opportunities, tasks, custom fields (nested under tasks/contacts/opportunities/custom keys)	Not modeled past Bronze in the provided data model; confirmed available and in scope (Section 4)
lead_activites_raw	Custom activities, SMS, calls, meetings, notes — nested under data: [ ]	Same activity re-appears in every daily batch → duplicates
custom_activites_raw	Custom activity type definitions, each with a nested field list mapping opaque field IDs (custom.cf_<hash>) to human field names — a two-level dictionary, not a flat outcome list	Reference data for resolving activity-level custom.cf_<hash> keys to readable field names (Section 5.3)
close_crm_users_raw	CRM user (setter/closer) profile and role data	Needed for setter/closer attribution

Certain field values — specifically, multi-select custom fields such as Objections Faced — are delivered as strings using single quotes instead of double quotes around a pseudo-JSON array, which breaks standard JSON parsers and must be repaired before the second-level parse (Section 5.1).

3. High-Level Architecture

The pipeline follows a Bronze → Silver → Gold → Export flow, entirely within Snowflake. Bronze preserves raw fidelity for replay; Silver repairs, flattens, deduplicates, and resolves business

attributes; Gold exposes business-logic views over Silver; Export materializes report-ready aggregates and unloads them to S3 for downstream consumption. This is a flatter shape than a classic dimensional star schema — Gold views compute report semantics directly rather than joining a separate dimension/fact layer, which matches the table and view structure already defined in the provided data model.

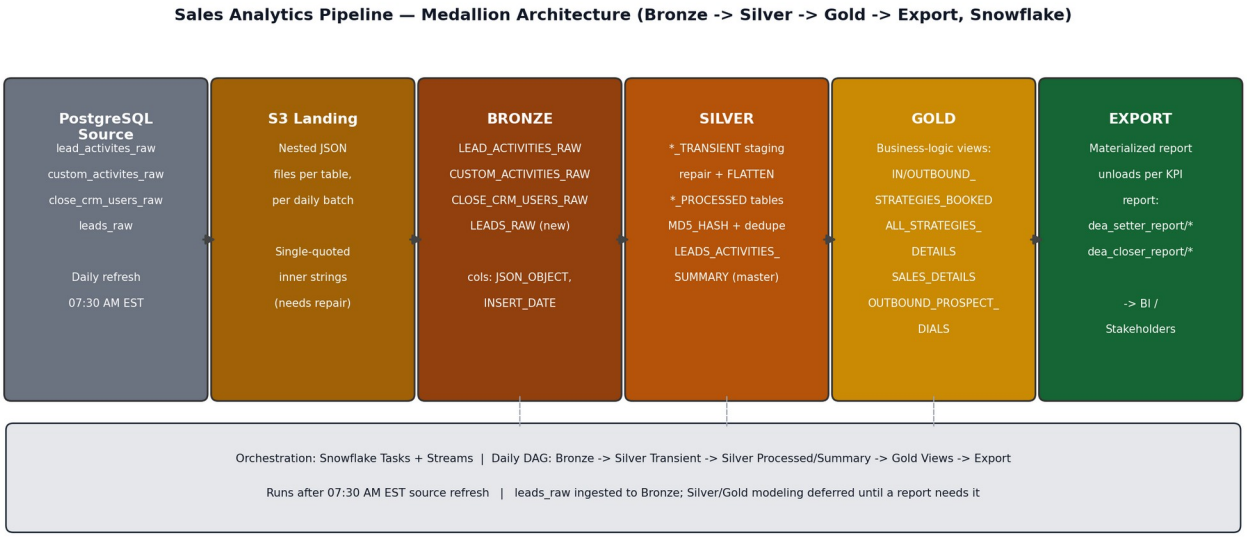


Figure 1. End-to-end flow from PostgreSQL source through S3 landing, Bronze, Silver, Gold, Export, with daily Snowflake Task orchestration.

4. Bronze Layer Design

Bronze applies no business logic — each daily JSON payload lands as-is behind a single VARIANT-typed column, so any record can be traced back to its source delivery and reprocessed if repair logic changes downstream. This matches the exact structure defined in the provided data model:

Bronze Table	Exact Column Structure
LEAD_ACTIVITIES_RAW	JSON_OBJECT, INSERT_DATE
CUSTOM_ACTIVITIES_RAW	JSON_OBJECT, INSERT_DATE
CLOSE_CRM_USERS_RAW	JSON_OBJECT, INSERT_DATE
LEADS_RAW (added — see decision below)	JSON_OBJECT, INSERT_DATE

Ingestion mechanics: as originally proposed, each daily extract would land in S3 in a per-table, date-partitioned prefix (for example, s3://<bucket>/raw/lead_activities/dt=2026-08-12/), loaded via a Snowflake external stage and COPY INTO. In the Phase 4 real-data build this automated path isn't available: the Snowflake account provisioned for this project is a trial account, which cannot create an External Access Integration, so there is no automated way to pull from Postgres into S3 on a schedule. Bronze ingestion is instead a manual step — data is exported from Postgres and loaded via PUT + COPY INTO into the internal stage @BRONZE.RAW_LANDING whenever a refresh is needed, rather than on a fixed daily schedule (see BRONZE.PURGE_REAL_DATA_STAGE for stage cleanup, itself also a manual step for the same reason — Section 8). Bronze remains append-only and the

JSON_OBJECT / INSERT_DATE structure is unchanged; only the delivery mechanism and cadence differ from the original proposal. Deduplication is still deferred to Silver so Bronze remains a complete, replayable log.

4.1 Design Decision: Including leads_raw

The provided data model's Bronze/Silver layers do not include leads_raw, and the four required KPI reports are fully served by activity-based data (LEADS_ACTIVITIES_SUMMARY and the Gold views built on it). Per SME guidance, leads_raw is available like any other source table, and it is this pipeline's call — as the design evolves — whether to bring it in. Decision: land LEADS_RAW in Bronze now, since ingestion cost is low and it preserves optionality, but defer Silver/Gold modeling of lead, contact, and opportunity detail until a concrete downstream need arises (for example, enriching reports with lead source/organization, or cross-checking CONTRACTED_VALUE in SALES_DETAILS against opportunity value). This keeps Phase 1 scope aligned to the four reports while not discarding data that later reporting may need.

5. Silver Layer Design

5.1 JSON Repair & Transient Staging

Nested string fields use single quotes (') instead of double quotes (") around JSON properties and values, which a strict parser rejects. A custom SQL UDF repairs each string before the second PARSE_JSON call by: (1) protecting apostrophes inside free-text fields such as notes or descriptions, so genuine contractions are not corrupted; (2) replacing the remaining structural single quotes with double quotes; and (3) restoring the protected apostrophes. Repaired-but-not-yet-merged output lands in transient staging tables that mirror the Bronze shape:

Transient Table	Exact Column Structure	Feeds
CUSTOM_ACTIVITIES_TRANSIENT	JSON_OBJECT, INSERT_DATE	CUSTOM_ACTIVITY_TYPES, CUSTOM_ACTIVITY_FIELDS
LEAD_ACTIVITIES_PROCESSED_TRANSIENT	JSON_OBJECT, INSERT_DATE	LEAD_ACTIVITIES_PROCESSED
LEADS_ACTIVITIES_SUMMARY_TRANSIENT	Same shape as LEADS_ACTIVITIES_SUMMARY (below)	LEADS_ACTIVITIES_SUMMARY

5.2 Deep Unwrapping & Activity Flattening

LATERAL FLATTEN unwraps the data: [ ] array from each raw payload into one row per activity. LEAD_ACTIVITIES_PROCESSED is the flattened, typed activity table:

Table	Exact Column Structure
LEAD_ACTIVITIES_PROCESSED	ACTIVITY_ID, LEAD_ID, CONTACT_ID, USER_ID, TYPE, TEXT, CREATED_BY_NAME, UPDATED_BY_NAME, ATTENDEES, CUSTOM_ACTIVITY_TYPE_ID, MEETING_STARTS_AT, MEETING_END_AT, MEETING_STATUS, ACTIVITY_AT, DATE_CREATED, DATE_UPDATED, CREATED_BY_USER, UPDATED_BY_USER, INSERT_DATE, UPDATE_DATE, MD5_HASH
LEAD_ACTIVITIES_EMAIL	LEAD_ID, ACTIVITY_ID, ACTIVITY_AT, DIRECTION, SUBJECT, BODY_TEXT, SENDER, STATUS, USER_ID, USER_NAME, TEMPLATE_ID, TEMPLATE_NAME, DATE_SENT, HAS_REPLY, ATTACHMENTS_JSON, ENVELOPE_JSON, SENDER_JSON, OPENS_JSON, JSON_PAYLOAD, DATE_CREATED, DATE_UPDATED, INSERT_DATE

LEAD_ACTIVITIES_EMAIL isolates SMS/email-type engagement with its own payload shape; it is available for future engagement reporting but is not a direct input to the four required KPI reports.

5.3 Custom Activity Field Resolution

Revised after reviewing the project's explanation video and transcript. Custom activity records do not carry human-readable field keys — each custom field appears as a top-level key named custom.cf_<hash> (an opaque Close-generated field ID, e.g. custom.cf_56b115c2...), and that ID only means something once resolved against custom_activites_raw's own dictionary. That dictionary is itself two levels: one CustomActivityType per activity type (Triage Call, Strategy Call, and so on), each carrying a nested fields array that maps its own field IDs to human names (Setter, Closer, Contract Value, "Triage Call Outcome", and so on). Silver mirrors that with two dictionary tables plus the resolved wrapper:

Table	Exact Column Structure
CUSTOM_ACTIVITY_TYPES	CUSTOM_ACTIVITY_TYPE_ID, CUSTOM_ACTIVITY_TYPE_NAME, MD5_HASH, INSERT_DATE, UPDATE_DATE
CUSTOM_ACTIVITY_FIELDS	CUSTOM_ACTIVITY_TYPE_ID, FIELD_ID, FIELD_NAME, FIELD_TYPE, MD5_HASH, INSERT_DATE, UPDATE_DATE
CUSTOM_ACTIVITIES_ALL_LEADS_DETAILS	LEAD_ID, ACTIVITY_ID, ACTIVITY_AT, WRAPPED_ACTIVITIES, INSERT_DATE, UPDATE_DATE, MD5_HASH

For each activity, every custom.cf_<hash> key is flattened out, looked up in CUSTOM_ACTIVITY_FIELDS by (activity type, field ID) to get its real name, and re-packaged into WRAPPED_ACTIVITIES — a semi-structured object keyed by human field name (Triage Call Outcome, Strategy Call Outcome, Offer Presented, Objections Faced, Contract Value, Cash Collected, and so on) rather than one column per field — so new custom fields on an existing activity type flow through automatically without a table alter (see Section 9). One field, Objections Faced, is multi-select: Close returns it as a single-quoted, Python-list-repr string rather than a real JSON array, which is what the single-quote JSON repair logic in Section 5.1 actually exists to fix (not a single blob covering every custom field, as an earlier draft of this design assumed).

5.4 User Attribution

Table	Exact Column Structure
CLOSE_CRM_USERS_PROCESSED	USER_ID, EMAIL, FIRST_NAME, LAST_NAME, MD5_HASH, DATE_CREATED, DATE_UPDATED, INSERT_DATE, UPDATE_DATE

Role (setter vs. closer) is resolved at the Gold layer based on which activity type/stage the user is attached to, rather than a static flag on the user record, since the same person can appear as a setter on one lead and a closer on another.

5.5 Change Data Capture & Deduplication

Because each daily batch re-appends the full history of a lead's activities, the same activity shows up in multiple batches. Every processed/summary Silver table carries an MD5_HASH column and applies:

- MD5_HASH — computed by concatenating key business attributes per row to detect real changes vs. re-delivery of unchanged rows.
- Duplicate key — lead_id + activity_id identifies the same logical activity across daily batches.
- Keep-latest rule — among duplicates, the row with the most recent activity_at is retained.
- Upsert — a MERGE INTO statement inserts new records and updates existing ones only when MD5_HASH has changed, maintaining INSERT_DATE and UPDATE_DATE audit columns.

5.6 Master Activity Table

LEADS_ACTIVITIES_SUMMARY is the fully resolved, one-row-per-activity table that everything in Gold reads from. It joins LEAD_ACTIVITIES_PROCESSED with CUSTOM_ACTIVITIES_ALL_LEADS_DETAILS (WRAPPED_ACTIVITIES), CUSTOM_ACTIVITY_TYPES for the activity type name, and CLOSE_CRM_USERS_PROCESSED for internal user identity. CUSTOM_ACTIVITY_OUTCOME_NAME is resolved per activity type (each type stores its outcome under a different WRAPPED_ACTIVITIES key — "Triage Call Outcome", "Strategy Call Outcome", and so on), since Close no longer ships a separate outcome-code field: the outcome text itself is the only identifier available.

Table	Exact Column Structure
LEADS_ACTIVITIES_SUMMARY	ACTIVITY_ID, LEAD_ID, CONTACT_ID, USER_ID, TYPE, TEXT, CREATED_BY_NAME, UPDATED_BY_NAME, ATTENDEES, CUSTOM_ACTIVITY_TYPE_ID, MEETING_STARTS_AT, MEETING_END_AT, MEETING_STATUS, ACTIVITY_AT, DATE_CREATED, DATE_UPDATED, CREATED_BY_USER, UPDATED_BY_USER, INSERT_DATE, UPDATE_DATE, WRAPPED_ACTIVITIES, CUSTOM_ACTIVITY, CUSTOM_ACTIVITY_OUTCOME_NAME, CUSTOM_ACTIVITY_OUTCOME, DEA_INTERNAL_NAME, DEA_INTERNAL_EMAIL

6. Gold Layer Design

Gold is implemented as business-logic views over LEADS_ACTIVITIES_SUMMARY (and CUSTOM_ACTIVITY_TYPES for activity-type context), each scoped to one stage or slice of the funnel, rather than a generic dimension/fact star schema. This keeps each view's grain and filter logic auditable against the exact business rules in the requirements.

6.1 Gold Views

View	Exact Column Structure
INBOUND_STRATEGIES_BOOKED	LEAD_ID, ACTIVITY_LOG_DATE, SETTER_CLOSER_EMAIL, SETTER_CLOSER_NAME, TRIAGE_CALL_OUTCOME, TRIAGE_CALL_DATE, STRATEGY_CALL_BOOKED, SC_YEAR_WEEK
OUTBOUND_STRATEGIES_BOOKED	LEAD_ID, ACTIVITY_LOG_DATE, SETTER_CLOSER_EMAIL, SETTER_CLOSER_NAME, PROSPECT_CALL_DATE, STRATEGY_CALL_BOOKED, SC_YEAR_WEEK
OUTBOUND_PROSPECT_DIALS	LEAD_ID, ACTIVITY_LOG_DATE, PROSPECT_YEAR_WEEK, CUSTOM_ACTIVITY, STATUS, CUSTOM_ACTIVITY_OUTCOME_NAME, CUSTOM_ACTIVITY_OUTCOME, SETTER_CLOSER_EMAIL, SETTER_CLOSER_NAME
ALL_STRATEGIES_DETAILS	LEAD_ID, ACTIVITY_LOG_DATE, CUSTOM_ACTIVITY, STATUS, CLOSER_EMAIL, CLOSER_NAME, STRATEGY_CALL_OUTCOME, DATE_OF_STRATEGY_CALL, OFFER_PRESENTED, SETTER, SETTER_NAME
SALES_DETAILS	LEAD_ID, ACTIVITY_AT, SALE_STATUS, SETTER_EMAIL, SETTER_NAME, CLOSER_EMAIL, CLOSER_NAME, CONTRACTED_VALUE, DATE_OF_SALE, PROGRAM, CASH_COLLECTED

SETTER/CLOSER on ALL_STRATEGIES_DETAILS and SALES_DETAILS resolve primarily from that activity's own "Setter"/"Closer" custom fields (captured directly on the Strategy Call / New Sale activity, per the explanation video), joined to CLOSE_CRM_USERS_PROCESSED — with a fallback to the original look-back-to-booking-view logic for any historical row where that field wasn't captured.

6.2 Funnel Logic

The funnel links initial contact → strategy call → sale per lead, in strict sequence, so a stage only counts once the prior stage is validly attended. This mirrors the worked examples in the requirements (Section 3) and is what each Gold view filters down to:

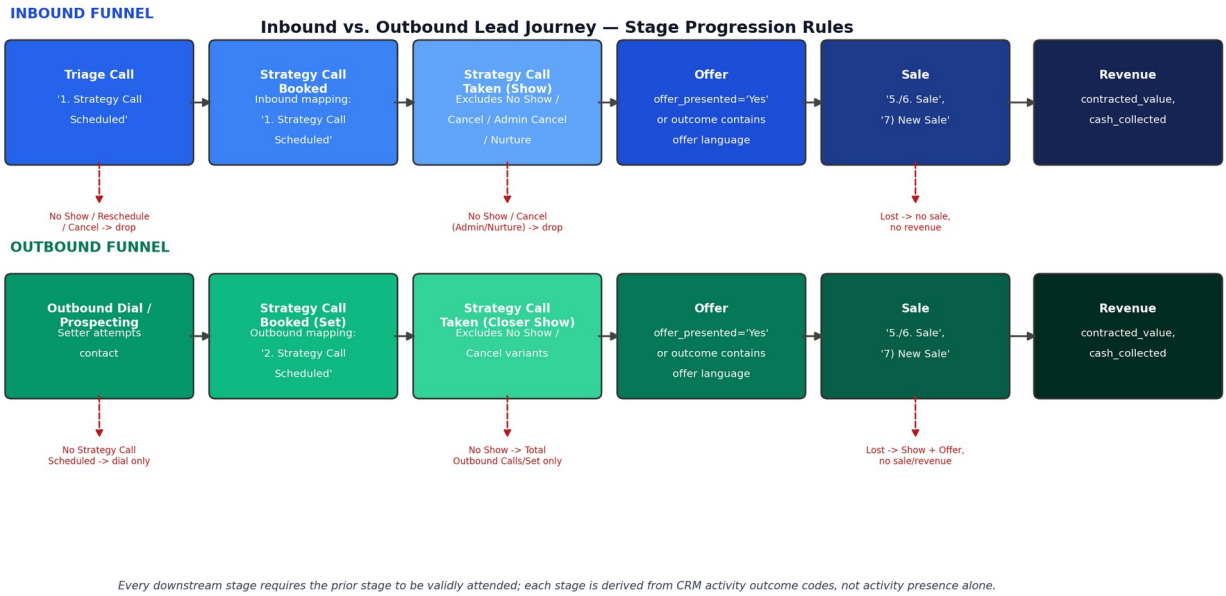


Figure 2. Inbound and outbound funnels share the same downstream stage logic but differ in how Initial Contact and Strategy Call Booked are qualified.

- Inbound Initial Contact is positively qualified only by the Triage Call outcome '1. Strategy Call Scheduled'; 'No Show', 'Reschedule', and 'Cancel' outcomes do not progress the lead — captured in INBOUND_STRATEGIES_BOOKED (TRIAGE_CALL_OUTCOME, STRATEGY_CALL_BOOKED).
- Outbound Initial Contact is positively qualified by the prospecting/outbound call outcome '2. Strategy Call Scheduled' — captured in OUTBOUND_PROSPECT_DIALS (dial-level detail) and rolled up in OUTBOUND_STRATEGIES_BOOKED.
- Strategy Call Taken (Show) excludes no-show and cancellation outcomes (Admin Cancel, Cancel – Not Interested, No Show, Cancel – Nurture, in their various numbered forms); only attended outcomes (Follow Up, Sale, Lost) count — evaluated from STRATEGY_CALL_OUTCOME in ALL_STRATEGIES_DETAILS.
- Offer is flagged in ALL_STRATEGIES_DETAILS.OFFER_PRESENTED when offer_presented = 'Yes' or the Strategy Call outcome text itself indicates an offer was made.
- Sale requires both an attended Strategy Call and a qualifying downstream sale activity ('5./6. Sale', '7) New Sale'); SALES_DETAILS is only populated for leads that cleared that bar, so a sale is never attributed without a valid attended conversation.

This is implemented by sequencing LEADS_ACTIVITIES_SUMMARY chronologically per LEAD_ID and evaluating the rules as an ordered state machine, so the Setter, Closer, and Objections reports all agree on which leads reached which stage.

6.3 Mapping Gold Views to the Four KPI Reports

Report	Primary Gold View(s)	Key Measures
Inbound Setter Report	INBOUND_STRATEGIES_BOOKED, ALL_STRATEGIES_DETAILS, SALES_DETAILS	Booked/Taken, Show Rate, Triage Set Rate, Offer Rate, Sale Rate, Average Order Value
Outbound Setter Report	OUTBOUND_PROSPECT_DIALS, OUTBOUND_STRATEGIES_BOOKED, ALL_STRATEGIES_DETAILS, SALES_DETAILS	Total Calls, Leads Touched, Outbound Set, Closer Show, Dial-to-Set / Set-to-Show / Show-to-Sale Rates
Closer Report	ALL_STRATEGIES_DETAILS, SALES_DETAILS	Call Booked, Cancellation Breakdown, No Show vs. Show, Lost vs. Sale, Avg. Contract Value, Cash Collected
Objections Faced Report	ALL_STRATEGIES_DETAILS (objection fields via WRAPPED_ACTIVITIES)	Total Calls Evaluated, Objection Category Count & %

7. Export Layer Design

A final Export step aggregates each Gold view down to the exact report grain and column list, then unloads the result to a dedicated S3 stage path per report — this is the artifact operational and management stakeholders actually consume, and what a future BI tool would point at directly.

Report	Target Stage	Exact Output Columns
Inbound Setter Report	dea_setter_report/inbound/	triage_date, setter, inbound_booked, inbound_taken, show_rate, triage_set_rate, strategy_call_booked, strategy_call_taken, offer_rate, total_sales, sale_rate, average_order_value
Outbound Setter Report	dea_setter_report/outbound/	dial_date, setter, total_outbound_calls, total_leads_touched, outbound_set, total_closer_show, total_offer, total_sale, dial_to_set_rate, set_to_show_rate, show_to_sale_rate, total_revenue, average_order_value
Closer Report	dea_closer_report/ closer_report/	CLOSER_NAME, call_year_month, CALL_BOOKED, ADMIN_CANCEL, CANCEL_NURTURE, CANCEL_NOT_INTEREST, TOTAL_CANCEL, NO_SHOW, STRTGY_CALL_SHW, LOST, sale, avg_vaue, cash_collected
Objections Faced Report	dea_closer_report/ objections_faced/	closer_name, activity_date, total_calls, money_count, fear_count, hung_up_count, logistical_count, no_obj_count, other_coaches_count, partner_count, think_abt_it_count, time_count, trust_count, value_count, not_looking_count, and the matching % column for each category

Grain differs by report — Inbound/Outbound Setter reports are daily by setter, the Closer and Objections reports roll up to call_year_month / activity_date by closer. This should be confirmed explicitly with the SME since it affects how SC_YEAR_WEEK and call_year_month are derived upstream.

8. Orchestration & Scheduling

Snowflake Tasks and Streams provide native orchestration without an external scheduler:

- In the Phase 4 real-data build, only Silver refresh and Export are on the scheduled Task DAG: TASK_REFRESH_SILVER_REAL runs daily (9:00 AM America/New_York) and calls SILVER.REFRESH_ALL_REAL_DATA(); TASK_EXPORT_REPORTS_REAL runs immediately after via an AFTER dependency and calls GOLD.EXPORT_ALL_REPORTS(). Bronze load is intentionally excluded from this DAG (see Section 4) since it's a manual, on-demand step rather than a fixed daily job.
- Rather than Snowflake Streams, SILVER.REFRESH_ALL_REAL_DATA() rebuilds each transient staging table in full from current Bronze on every run, then reconciles into the processed/summary tables via MERGE INTO — a full-scan upsert rather than incremental stream-based CDC. At current data volumes this runs well within the daily window; Streams remain a reasonable optimization if source volume grows enough to make full rebuilds slow.
- INSERT_DATE / UPDATE_DATE columns, together with Stream offsets, prevent reprocessing files that were already loaded.
- Stage purge is a separate, manual procedure (BRONZE.PURGE_REAL_DATA_STAGE) rather than an automatic date-pattern purge tied to the daily DAG. Real-data files have no date-partitioned naming convention the way the original daily S3 extracts would have, and auto-purging on the Silver refresh schedule risks deleting a file mid-upload if a manual Bronze refresh happens to overlap a scheduled run. This is run manually, right after confirming a fresh COPY INTO succeeded.
- Task failures alert via Snowflake's task error notifications so a failed daily run is caught before it silently breaks downstream reports.

9. Schema Evolution & Data Quality

- Bronze stores payloads as JSON_OBJECT (VARIANT), so new or changed source fields land without breaking ingestion.
- WRAPPED_ACTIVITIES stores resolved custom fields as a semi-structured object rather than one column per field, so new fields on an existing custom activity type flow through automatically once they appear in custom_activites_raw's field list; a genuinely new activity type still requires a CUSTOM_ACTIVITY_TYPES row and an SME-reviewed addition to the funnel logic (e.g. a new WHEN branch in the outcome-resolution CASE statement) before it affects Gold.
- Data quality checks: row-count reconciliation between Bronze and Silver, null-rate checks on key business fields (LEAD_ID, ACTIVITY_ID, ACTIVITY_AT), and orphan detection for activities referencing unknown leads or users.

10. Assumptions & Open Items for SME Review

Resolved: leads_raw is confirmed available and in scope; per SME guidance the provided data model is a guide, not a fixed spec, and the pipeline design is expected to evolve as needed. See Section 4.1 for the resulting decision (ingest to Bronze now, defer Silver/Gold modeling until a report needs it).

Resolved against real data (Phase 4): three further items from this list have since been confirmed by loading and inspecting the actual PostgreSQL / Close CRM data.

- Full, authoritative outcome-code lists per activity type: confirmed by loading all 28 real CUSTOM_ACTIVITY_TYPES and 226 CUSTOM_ACTIVITY_FIELDS records from Close CRM. Gold funnel logic (Section 6.2) now matches exact real outcome strings rather than the requirements-document examples.
- Role/status fields on close_crm_users_raw: confirmed absent from the real object. A sampled real user record contains only date_created, date_updated, email, email_verified_at, first_name, google_profile_image_url, id, image, last_name, last_used_timezone, and organizations. CLOSE_CRM_USERS_PROCESSED already captures every field that actually exists, and Section 5.4's Gold-layer role resolution is confirmed as the only viable approach for this data, not merely a design preference.
- Sale-deriving activity types: requirements Section 3.4 specifies two — '7) New Sale' and '8) New Sale [Custom Payment Plan]'. The Custom Payment Plan type has no Contract Signed or Cash Collected fields; it instead uses an installment structure (Initial Payment, Recurring Amount, Number of Billing Cycles). SALES_DETAILS now covers both types, falling back to Initial Payment for cash collected and treating the activity's existence as an implicit Contract Signed = Yes where that field is absent.

The following remain open and should be confirmed with the SME before Phase 2/3 begin:

- The single-quote JSON repair approach covers all malformed patterns present in production data, not just the sampled examples.
- ACTIVITY_AT is reliably populated across all activity types and is safe to use for both sequencing and MD5-based dedup.
- Timezone handling: source refresh is anchored to 07:30 AM EST; confirm ACTIVITY_AT, ACTIVITY_LOG_DATE, and the report date/week/month fields are all normalized to the same timezone.
- Confirm the source of PROGRAM in SALES_DETAILS and the exact grain/definition of SC_YEAR_WEEK and call_year_month used for weekly/monthly rollups in the Export reports.

11. Next Steps

- Phase 2 — Produce the detailed ERD for Bronze, Silver, Gold, and Export, reconciled against this document and the provided Snowflake Data Architecture Model, including LEADS_RAW in Bronze.
- Phase 3 — Build Bronze COPY INTO pipelines, Silver repair/flatten/MERGE logic, Gold view SQL and funnel logic, Export unload jobs, and Task/Stream orchestration; run daily for 7 days to simulate production before handoff.